PROJET DE RECHERCHE D'INFORMATION

Phase 1

Interface administrateur d'indexation dynamique

Rapport technique et fonctionnel

Établissement / formation	À compléter
Module	Recherche d'information
Niveau	Ingénierie en Intelligence Artificielle — Semestre 9
Étudiant(s)	À compléter
Enseignant	À compléter
Année universitaire	À compléter

Document de présentation du travail réalisé, de ses choix techniques et de ses limites.

Résumé

Ce rapport présente la réalisation de la première phase d'un moteur de recherche d'information. L'application fournit à l'administrateur une interface graphique pour charger des articles Reuters au format SGML, configurer un pipeline de prétraitement, construire des représentations BoW, F, TF et TF-IDF, comparer leur occupation mémoire et sauvegarder les index pour une réutilisation ultérieure. Le projet est organisé en modules Python afin de séparer l'interface, le chargement, le traitement linguistique, l'indexation et la persistance.

À retenir : les règles de stopwords, de racinisation et de lemmatisation sont implémentées manuellement à un niveau pédagogique. La lemmatisation est une approximation fondée sur des suffixes et quelques exceptions; elle ne remplace pas un analyseur linguistique complet. Le comptage matriciel s'appuie sur scikit-learn et les calculs numériques sur NumPy.

Sommaire

Partie	Contenu
1	Contexte, objectifs et périmètre
2	Architecture logicielle
3	Parcours utilisateur
4	Prétraitement linguistique
5	Modèles et structures d'indexation
6	Mesure mémoire et sauvegarde
7	Vérifications effectuées
8	Limites connues et perspectives
9	Installation, lancement et démonstration

1. Contexte, objectifs et périmètre

La phase 1 porte sur le module administrateur. Son rôle est de préparer les documents et les structures nécessaires à une future recherche par requête. Le système est conçu pour travailler sur un sous-ensemble du corpus Reuters-21578, sélectionné depuis le disque local. Un nombre maximal de documents peut être défini afin de faciliter les essais.

Objectifs fonctionnels

Besoin	Réponse apportée par l'application
Sélection des données	Choix d'un dossier local contenant des fichiers .sgm.
Prétraitement paramétrable	Étapes activables et réordonnables dans l'interface.
Indexation	Calcul des quatre modèles et création des formats matriciels et inversés.
Suivi	Affichage du nombre de documents, du vocabulaire, du temps et des tailles mémoire.
Persistance	Écriture de index.pkl et config.json dans le dossier choisi.

La phase 2 (recherche utilisateur, calcul de similarité et classement Top-K) n'est pas implémentée dans ce livrable. L'aperçu d'un terme dans la liste inversée sert uniquement à vérifier l'index produit.

2. Architecture logicielle

Le programme est séparé en fichiers dont les responsabilités sont volontairement simples. Le point d'entrée lance l'interface; les calculs sont confiés à des modules distincts, ce qui facilite la lecture et la réutilisation.

Fichier	Responsabilité
main.py	Crée l'objet Application et démarre la boucle graphique Tkinter.
gui.py	Présente les contrôles, appelle les modules et affiche les résultats ou erreurs.
chargement.py	Parcourt le dossier, lit les fichiers SGML et extrait l'identifiant, le titre et le corps.
pretraitement.py	Contient les cinq fonctions de traitement et l'exécution ordonnée du pipeline.
indexation.py	Construit les fréquences, les quatre modèles, les matrices et les listes inversées.
stockage.py	Écrit et relit les fichiers index.pkl et config.json.
rapport_final.py	Produit ce rapport PDF à partir du code du projet.

Données Reuters	Chargement	Pipeline NLP	Indexation	Affichage / sauvegarde
fichiers .sgm	textes par ID	tokens par document	matrices + postings	métriques, .pkl, .json

Le flux est séquentiel : les textes bruts deviennent des tokens; les tokens alimentent le calcul des matrices; les matrices alimentent les listes inversées et les statistiques. L'interface orchestre ce flux mais ne contient pas les règles mathématiques principales.

3. Parcours utilisateur

Étape	Action et résultat
1. Charger	Choisir un dossier contenant des fichiers .sgm. L'application indique le nombre d'articles extraits.
2. Configurer	Cocher ou décocher une étape et utiliser les flèches pour changer l'ordre.
3. Générer	Lancer le prétraitement et la construction des index; un message signale le succès ou une erreur.
4. Examiner	Lire le nombre de documents, la taille du vocabulaire, le temps et le tableau mémoire; tester un mot.
5. Sauvegarder	Choisir le dossier de destination. L'application affiche les chemins exacts des deux fichiers créés.

La sauvegarde doit être faite après la génération. Si les paramètres du pipeline sont modifiés et que les index sont régénérés, il faut sauvegarder à nouveau pour conserver une configuration qui correspond aux index.

4. Prétraitement linguistique

Le pipeline transforme le texte en une suite de mots normalisés. Chaque étape reçoit une chaîne et renvoie une chaîne; le découpage en tokens est réalisé à la fin. L'ordre est important : par exemple, les stopwords sont plus facilement reconnus après la mise en minuscules.

Étape	Fonctionnement	Exemple / remarque
Tag cleaning	Retire les balises entre chevrons avec une expression régulière.	devient du texte séparé.
Normalisation	Met en minuscules et remplace ponctuation et chiffres par des espaces.	Bank et bank deviennent identiques.
Stopwords	Filtre les tokens présents dans un ensemble anglais écrit dans le code.	the, is, at sont supprimés.
Stemming	Applique des règles simples de suffixes.	Une terminaison comme ing peut être retirée.
Lemmatisation	Applique des règles de suffixes et quelques correspondances irrégulières.	Approche pédagogique, pas une analyse lexicale complète.

Choix “from scratch” : le jeu de stopwords et les règles de stemming/lemmatisation sont codés dans le projet sans NLTK. Le traitement SGML utilise la bibliothèque standard re. Ces règles sont volontairement accessibles à expliquer, mais moins linguistiquement précises que des outils spécialisés.

5. Modèles et structures d'indexation

L'indexation transforme le corpus en nombres. On note N le nombre de documents, M la taille du vocabulaire, F(t,d) le nombre d'occurrences du terme t dans le document d, et df(t) le nombre de documents contenant t.

Modèle	Définition utilisée	Interprétation
BoW	1 si F(t,d) > 0, sinon 0.	Présence ou absence du terme.
F	F(t,d).	Nombre brut d'occurrences.
TF	F(t,d) / max_t F(t,d).	Fréquence ramenée au maximum du document.
TF-IDF	TF(t,d) × log(N / df(t)).	Réduit le poids des termes présents dans de nombreux documents.

Le vocabulaire et les fréquences sont obtenus avec CountVectorizer configuré pour découper les tokens déjà préparés sur les espaces. NumPy sert à créer les tableaux numériques et à appliquer les formules. Le TF-IDF est calculé explicitement dans le code avec la formule non lissée log(N/df).

Formats produits pour chaque modèle

Format dans le code	Organisation	Utilité
matrice_terme_doc	Documents en lignes, termes en colonnes; dimensions N × M.	Accès direct au poids d'un terme pour un document.
matrice_doc_terme	Termes en lignes, documents en colonnes; dimensions M × N.	Même information transposée pour l'autre orientation demandée.
liste_inversee	Dictionnaire terme → [(id_document, poids), ...].	Ne conserve que les documents où le terme a un poids non nul.

Les noms des deux matrices sont conservés tels qu'ils figurent dans le programme. Le point important pour leur lecture est l'orientation explicitée ci-dessus.

6. Mesure mémoire et sauvegarde

Les tableaux NumPy sont convertis en float32; chaque case occupe donc quatre octets. La taille d'une matrice est calculée avec son nombre d'octets de données. Pour la liste inversée, le programme additionne la taille rapportée par sys.getsizeof pour le dictionnaire, les termes, les listes et les tuples.

Interprétation : la mesure de la liste inversée est une estimation de la mémoire des conteneurs Python, pas une mesure complète de la mémoire du processus. Les objets référencés (notamment les nombres) ne sont pas tous comptabilisés séparément. Le tableau sert donc à comparer les structures avec prudence, pas à présenter une mesure système exacte.

Fichier	Contenu	Lecture
index.pkl	Dictionnaire Python contenant vocabulaire, IDs, matrices et listes inversées.	Binaire; à recharger avec stockage.charger_index, pas avec un éditeur texte.
config.json	Étapes utilisées, nombre de documents, taille du vocabulaire et date de création.	Texte lisible; peut être ouvert dans un éditeur.

Les deux fichiers doivent rester ensemble dans le même dossier. Le code fournit une fonction de rechargement Python, mais l'interface graphique actuelle n'a pas de bouton pour importer un index déjà sauvegardé.

7. Vérifications effectuées

Les vérifications de développement ont porté sur la compilation des fichiers Python et sur un parcours réduit avec le corpus d'exemple fourni dans test_docs.

Vérification	Résultat observé
Compilation des modules Python	Réussie avec python -m py_compile.
Chargement du corpus d'exemple	5 documents extraits du fichier fourni lors du test.
Pipeline et indexation	Vocabulaire et quatre modèles produits; le terme oil est présent dans l'index inversé TF-IDF.
Sauvegarde puis rechargement	Cycle index.pkl/config.json testé dans un dossier temporaire; IDs et étapes récupérés.

Ces résultats constituent des tests fonctionnels sur un petit échantillon. Ils ne représentent pas un benchmark complet du corpus Reuters-21578 ni une certification de performance sur de très grands volumes.

8. Limites connues et perspectives

Limite identifiée	Conséquence / amélioration possible
Règles NLP simplifiées	Certains mots peuvent être mal racinisés ou lemmatisés; utiliser un analyseur plus complet si la précision linguistique devient prioritaire.
Dépendances NumPy et scikit-learn	Le NLP est manuel, mais le comptage vectoriel et les tableaux numériques reposent encore sur ces bibliothèques.
Matrices denses	La mémoire augmente avec N × M; une représentation creuse serait préférable pour un grand corpus.
Pas d'import d'index depuis l'interface	Le rechargement existe au niveau du module stockage, mais doit être déclenché par du code Python.
Pas de recherche de phase 2	L'aperçu d'un mot n'est pas un moteur de classement de documents par similarité.
Mesure mémoire approximative	Elle compare les structures mais ne mesure pas toute la mémoire réellement consommée par le processus.

9. Installation, lancement et démonstration

Prérequis

Python 3.8 ou supérieur, avec les dépendances NumPy et scikit-learn. Depuis PowerShell, dans le dossier du projet, installer au besoin avec :

```
python -m pip install numpy scikit-learn
```

Démarrage et démonstration

Depuis le dossier projet, lancer :

```
python main.py
```

Ordre	Démonstration proposée à l'enseignant
1	Choisir test_docs, vérifier le nombre d'articles chargés.
2	Montrer les cinq étapes et modifier leur ordre.
3	Générer les index et commenter BoW, F, TF et TF-IDF.
4	Comparer les tailles affichées et rechercher un terme comme oil.
5	Sauvegarder dans un dossier facile à retrouver et montrer index.pkl avec config.json.

Pour remettre le travail, joindre le code source, le corpus d'exemple utilisé pour la démonstration, le rapport PDF et, si une sauvegarde est demandée, index.pkl avec config.json. Le fichier .pkl est binaire et doit être lu par le programme.

Conclusion

La phase 1 fournit un outil administrateur fonctionnel pour préparer un corpus, choisir un pipeline, construire plusieurs modèles d'indexation, comparer leurs structures et enregistrer les résultats. Le découpage modulaire et les règles NLP manuelles rendent le travail explicable en consultation. Les limites présentées dans ce rapport délimitent clairement le travail réalisé et les améliorations possibles avant la phase 2.

Rapport généré le 07/11/2026.